UNICESUMAR

GABRIEL DA SILVA VIEIRA DA ROSA, MATHEUS NOGUEIRA, PATRICK GUILHERME REGA GUIS

AVALIAÇÃO PRÁTICA – ESTRUTURA DE DADOS II

ESTRUTURAS, PESQUISA E ORDENAÇÃO DE DADOS

Ponta Grossa - PR
2026

GABRIEL DA SILVA VIEIRA DA ROSA, MATHEUS NOGUEIRA, PATRICK GUILHERME REGA GUIS

AVALIAÇÃO PRÁTICA – ESTRUTURA DE DADOS II

ESTRUTURAS, PESQUISA E ORDENAÇÃO DE DADOS

Relatório técnico apresentado como requisito parcial para avaliação da disciplina de Estruturas, Pesquisa e Ordenação de Dados. O presente trabalho tem caráter acadêmico e tem como finalidade a aplicação prática dos conteúdos da matéria em três projetos.

Orientador: Prof. MSc. Gabriel Passos de Jesus

Ponta Grossa - PR
2026

Sumário

1 Introdução Geral 4
2 Projeto I - Árvores 5
2.1 Introdução 5
2.2 Fundamentação Teórica 5
2.3 Análise Assintótica 5
2.4 Metodologia Experimental 6
2.5 Resultados 6
2.6 Discussão 6
2.7 Conclusão 6
3 Projeto II - Buscas 8
3.1 Introdução 8
3.2 Fundamentação Teórica 8
3.3 Análise Assintótica 8
3.4 Metodologia Experimental 9
3.5 Resultados 9
3.6 Discussão 9
3.7 Conclusão 10
4 Projeto III - Ordenação 11
4.1 Introdução 11
4.2 Fundamentação Teórica 11
4.3 Análise Assintótica 11
4.4 Metodologia Experimental 11
4.5 Resultados 12
4.6 Discussão 12
4.7 Conclusão 12
5 Conclusão Geral 13

1 Introdução Geral

O desenvolvimento de três projetos práticos foi realizado nesse trabalho, focando em diferentes abordagens relacionadas à manipulação de dados, como estruturas arbóreas, algoritmos de busca e algoritmos de ordenação, onde cada projeto teve o propósito de, além de implementar os algoritmos, também analisar seu comportamento sob diferentes cenários, considerando aspectos teóricos e experimentais.

A realização destes projetos permitiu a consolidação de alguns conceitos da disciplina, como a complexidade assintótica, a análise de desempenho e a metodologia experimental, por exemplo, sendo possível observar na prática como os diferentes tipos de algoritmo se comportam diante de variações nos dados de entrada.

2 Projeto I - Árvores

2.1 Introdução

Este projeto tem como objetivo desenvolver, testar e analisar três estruturas de dados arbóreas: a Árvore Binária de Busca (BST), a Árvore AVL e a Árvore Rubro-Negra. A análise foca no desempenho prático e teórico de cada estrutura em operações obrigatórias de manipulação: inserção, remoção, busca e cálculo de altura. Além das árvores, o projeto engloba a implementação de uma heurística aproximada para o Problema do Caixeiro-Viajante (PCV), visando aplicar e avaliar lógicas algorítmicas em problemas complexos de otimização.

2.2 Fundamentação Teórica

Árvore Binária de Busca (BST): Estrutura baseada em nós onde cada elemento possui até dois filhos. Valores menores ficam na subárvore à esquerda e maiores à direita. Sua principal fraqueza é a falta de balanceamento automático.

Árvore AVL: É uma BST auto-balanceável. Nela, a diferença de altura entre as subárvores esquerda e direita de qualquer nó (fator de balanceamento) é no máximo 1. Rotações são realizadas para manter essa regra.

Árvore Rubro-Negra: Outra BST auto-balanceável que utiliza um sistema de "cores" (vermelho ou preto) nos nós para garantir um balanceamento aproximado. Geralmente realiza menos rotações que a AVL durante inserções e remoções.

Problema do Caixeiro-Viajante (PCV): Problema clássico de rotas (classificado como NP-difícil) que busca o menor caminho para visitar um conjunto de cidades e voltar à origem. Utiliza-se heurísticas para encontrar boas soluções (aproximadas) em tempo computacional viável.

2.3 Análise Assintótica

Algoritmo	Médio Caso	Melhor Caso	Pior Caso
Árvore Binária de Busca (BST)	O(log n)	O(1)	O(n)
Árvore AVL	O(log n)	O(1)	O(log n)
Árvore Rubro-Negra	O(log n)	O(1)	O(log n)

Observação: O pior caso da BST clássica ocorre quando os dados são inseridos já ordenados, transformando a árvore em uma "lista" (degenerada).

As árvores AVL e Rubro-Negra evitam isso com o balanceamento.

2.4 Metodologia Experimental

Para realizar as comparações, os algoritmos foram testados em três volumes distintos de dados (tamanhos de entrada). A fim de garantir a validade e a precisão da análise estatística, foram executadas 30 repetições para cada experimento. Os testes mediram o tempo de execução (utilizando funções de cronometragem de alta precisão) para dados gerados de forma aleatória e também para dados já ordenados. O mesmo critério de repetições e variação de entradas foi aplicado aos experimentos do Caixeiro-Viajante.

2.5 Resultados

Nos experimentos com entradas aleatórias, as três árvores mostraram resultados consistentes e eficientes. O cenário mudou drasticamente ao utilizar entradas ordenadas: a Árvore Binária de Busca apresentou uma severa degradação de performance, elevando seu tempo de execução de forma linear. Por outro lado, a Árvore AVL e a Árvore Rubro-Negra mantiveram a velocidade e estabilidade, com a Rubro-Negra se mostrando ligeiramente mais rápida nas operações de inserção contínua devido à menor necessidade de rotações estruturais.

2.6 Discussão

Os dados obtidos na prática ratificam os conceitos da análise assintótica. A BST simples demonstrou ser perigosa para aplicações reais devido ao risco de perda total de eficiência no pior caso. A árvore AVL provou ser uma excelente estrutura para leituras e buscas graças ao seu formato estritamente balanceado. Já a árvore Rubro-Negra destacou-se pelo equilíbrio geral, sendo altamente recomendada para sistemas que sofrem frequentes alterações (inserções e remoções).

2.7 Conclusão

A conclusão extraída deste projeto reforça o impacto direto que o balanceamento estrutural possui no desempenho de sistemas computacionais. Fica evidente que árvores como a AVL e a Rubro-Negra são indispensáveis quando o software necessita de estabilidade e previsibilidade no tempo de resposta (O(log n)), independentemente

de como os dados entram no sistema. Paralelamente, os experimentos com o Caixeiro-Viajante demonstraram a utilidade prática das heurísticas, permitindo resolver problemas que de outra forma seriam computacionalmente impossíveis em larga escala.

3 Projeto II - Buscas

3.1 Introdução

Esse projeto tem como objetivo desenvolver, testar e analisar três algoritmos de buscas: Busca Sequencial, Busca Binária e Busca em Árvore de Busca Binária.

A análise considera a teoria e experimental dos algoritmos, medindo o desempenho em diferentes tamanhos de dados, utilizando métricas estatísticas para comparar os tempos de execução.

3.2 Fundamentação Teórica

Busca Sequencial: Consiste em percorrer uma lista do início ao fim até encontrar o valor esperado ou o final da lista.

Busca Binária: Precisa que a lista esteja ordenada. Consiste em “dividir” uma lista ao meio à cada iteração para diminuir o volume de dados e encontrar o resultado de forma rápida e eficiente.

Busca em Árvore de Busca Binária: Utiliza da estrutura hierárquica da árvore de busca binária, onde valores menores são organizados à esquerda e maiores à direita, o que deixa as buscas extremamente rápidas e eficientes.

3.3 Análise Assintótica

Algoritmo	Médio Caso	Melhor Caso	Pior Caso
Busca Sequencial	O(n)	O(1)	O(n)
Busca Binária	O(log n)	O(1)	O(log n)
Busca em Árvore Balanceada*	O(log n)	O(1)	O(log n)
Busca em Árvore Desbalanceada*	O(n)	O(1)	O(n)

Observações: Uma Árvore Balanceada é quando a árvore distribui bem seus elementos, mantendo uma altura curta, sem muitas verificações recursivas.

Já a Árvore Desbalanceada é quando ela vira uma “lista”, elementos ordenados inseridos um a seguir do outro, impedindo o uso da estrutura hierárquica da Árvore.

3.4 Metodologia Experimental

Os experimentos foram realizados com volumes diferentes de dados (n), para avaliar os algoritmos em diferentes cenários.

Para cada entrada, foram realizadas buscas utilizando os três algoritmos, assim como múltiplas repetições para cada teste.

O tempo de execução foi medido com a função “perf_counter” da biblioteca time em Python, para garantir maior precisão.

Com os dados obtidos, foram calculados as médias dos tempos de execução e o desvio padrão.

3.5 Resultados

Volume de Dados	Média Sequencial	Média Binária	Média Árvore
n=5000	0.000043s	0.000003s	0.000001s
n=50000	0.002585s	0.000002s	0.000002s
n=500000	0.051848s	0.000017s	0.000004s

Foi observado que, com o aumento da quantidade de dados, a Busca Sequencial possui crescimento linear com base no volume de dados integrados à ela. Já a Busca Binária e em Árvore, conseguem manter um resultado estável sem crescimento absurdo em seus casos médios.

3.6 Discussão

Os resultados e testes confirmaram o que já era esperado dos algoritmos.

Busca Sequencial: Apresenta maior tempo de execução dependendo do volume de dados, por precisar percorrer a lista inteira do início ao fim.

Busca Binária: Apresentou bons resultados com alta eficiência, mantendo um tempo de execução baixo mesmo com grandes volumes de dados.

Busca em Árvore de Busca Binária: Apresentou ótimos resultados na maioria das situações, onde seu pior caso é quando a árvore acaba virando uma lista ordenada (desbalanceada), causando resultados parecidos com a busca sequencial.

3.7 Conclusão

Em resumo, foi concluído que a escolha final do algoritmo de busca em um sistema impactará bastante o seu tempo de execução e responsividade geral.

A busca sequencial, enquanto simples, é extremamente ineficiente com grandes escalas de dados.

A busca binária provou ser uma escolha muito sólida em diversas situações, entretanto requer que a lista seja previamente ordenada para que funcione, o que talvez ocasione em um tempo de execução maior no final.

A busca em árvore binária de busca mostrou ser estável na grande maioria das situações, sendo ineficiente apenas em algumas específicas, como por exemplo quando a inserção é ordenada.

4 Projeto III - Ordenação

4.1 Introdução

Os algoritmos de ordenação são utilizados em diversas aplicações, como organização de dados, otimização de buscas e processamento de informações, sendo fundamentais na ciência da computação, porém, a velocidade e a eficiência desses algoritmos variam significativamente entre si e dependendo do tamanho e a organização dos dados de entrada.

Visando realizar esse teste, foram implementados 2 algoritmos de ordenação neste projeto, o Insertion Sort e o Quick Sort e com eles, o objetivo foi analisar o desempenho desses algoritmos em vários cenários, considerando o melhor, pior e médio caso.

4.2 Fundamentação Teórica

Insertion Sort: funciona semelhante a organização de cartas em uma mão. Percorre a lista e insere ada elemento na posição correta em relação aos elementos já ordenados e é mais eficiente em listas pequenas ou parcialmente ordenadas, apresentando um bom desempenho no melhor caso.

Quick Sort: é o algoritmo mais eficiente na maioria dos casos, pois utiliza de uma estratégia de divisão e conquista, onde seleciona um elemento como pivô e particiona a lista em 2 subconjuntos, elementos maiores e menores q o pivô e aplica o processo recursivamente a cada subconjunto.

4.3 Análise Assintótica

Algoritmo	Melhor Caso	Caso Médio	Pior Caso
Insertion Sort	O(n)	$O(N^2)$	$O(n^2)$
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$

O Insertion Sort apresenta desempenho linear no melhor caso, quando os dados já estão ordenados. Já o Quick Sort possui desempenho médio mais eficiente, mas pode atingir complexidade quadrática em cenários desfavoráveis.

4.4 Metodologia Experimental

Os experimentos foram realizados utilizando-se de diferentes tamanhos de entrada, 1000, 5000 e 10000 e com 3 cenários distintos, o melhor, médio e pior caso. O melhor

caso seria com a lista previamente ordenada, o médio com a lista aleatória e o por caso, com a lista de forma inversa, lista essa que foi ordenada de forma aleatória utilizando-se da biblioteca “random” e do “random.seed(42)”, que cria uma seed fixa e garante a reprodutibilidade dos experimentos. Já o tempo de execução de cada algoritmo foi calculado pela função perf_counter, da biblioteca padrão do Python.

4.5 Resultados

O Insertion Sort apresentou um desempenho satisfatório apenas nas entradas pequenas ou que já estavam ordenadas, enquanto o Quick Sort demonstrou um melhor desempenho na maioria dos casos, tendo também um tempo de execução que cresce de forma menos gradativa comparado ao do Insertion Sort, que aumenta de maneira significativa à medida em que o tamanho de entrada aumenta.

4.6 Discussão

Com base nos resultados, confirmaram-se as previsões teóricas dos algoritmos estudados, o Insertion Sort se demonstrou eficiente no melhor caso, mas apresentou uma baixa escalabilidade em cenários mais complexos. Já o Quick Sort, demonstrou melhor desempenho geral, mas também mostrou que a escolha inadequada do pivô pode levar à degradação do desempenho e até mesmo estourar o limite de recursão do Python. Devido a essa degradação, foi necessário ajustar a escolha do pivô durante os testes, que nem passavam de 800 casos devido a recursão excessiva.

4.7 Conclusão

Com esse projeto, foi possível compreender o funcionamento e o desempenho de cada um dos algoritmos de ordenação, observando-se que algoritmos simples, como o Insertion Sort, podem ser eficientes em casos específicos, mas em casos maiores e mais complexos, perde muita eficiência, enquanto o contrário ocorre com o Quick Sort, que tem um desempenho geral melhor devida a sua maior complexidade.

Além disso, foi possível perceber como a escolha do algoritmo impacta na performance geral do sistema, demonstrando a importância da escolha correta, e, no caso do Quick Sort, da estratégia adequada de implementação, como a seleção do pivô, que pode impactar diretamente no desempenho geral do sistema.

5 Conclusão Geral

A realização destes 3 projetos nos uma compreensão prática dos principais conceitos abordados na disciplina de Estruturas de Dados, demonstrando a importância da escolha adequada de estruturas e algoritmos para a resolução eficiente dos problemas computacionais.

No projeto I, foi possível observar o impacto do balanceamento em estruturas arbóreas. As árvores AVL e Rubro-Negra demonstraram uma maior estabilidade e eficiência comparadas a Árvore Binária, garantindo operações eficientes independentemente da ordem da inserção dos dados.

No projeto II, a análise dos algoritmos de busca evidenciou diferenças significativas de desempenho entre as abordagens, pois a busca binária apresentou um excelente desempenho se aplicada a dados previamente ordenados, enquanto a busca sequencial mostrou-se simples, porém ineficiente em grandes volumes de dados, enquanto a busca em árvore destacou a dependência do balanceamento da estrutura, visando manter eficiência.

No projeto III, os algoritmos foram analisados sob diferentes cenários, permitindo uma observação e comparação prática da relação de complexidade teórica e desempenho real. O Insertion Sort demonstrou eficiência em casos específicos, enquanto o Quick Sort apresentou melhor desempenho geral, embora sensível à escolha do pivô, o que pode impactar negativamente sua execução em determinados casos.

Os resultados obtidos demonstram que a eficiência de um algoritmo não depende somente de sua complexidade teórica, mas também de características dos dados de entrada e da forma como o algoritmo foi implementado.